Pearls AQI Predictor

Final Project Report

Prepared by: Ali Jawad

Submitted To: 10 Pearls shine internship Program

Department: Data Science

Live Dashboard: https://shineaqipredictor.streamlit.app/

Code Repository: https://github.com/alijawad480/pearls-aqi-predictor

Email: alijawadofficial480@gmail.com

1. Introduction

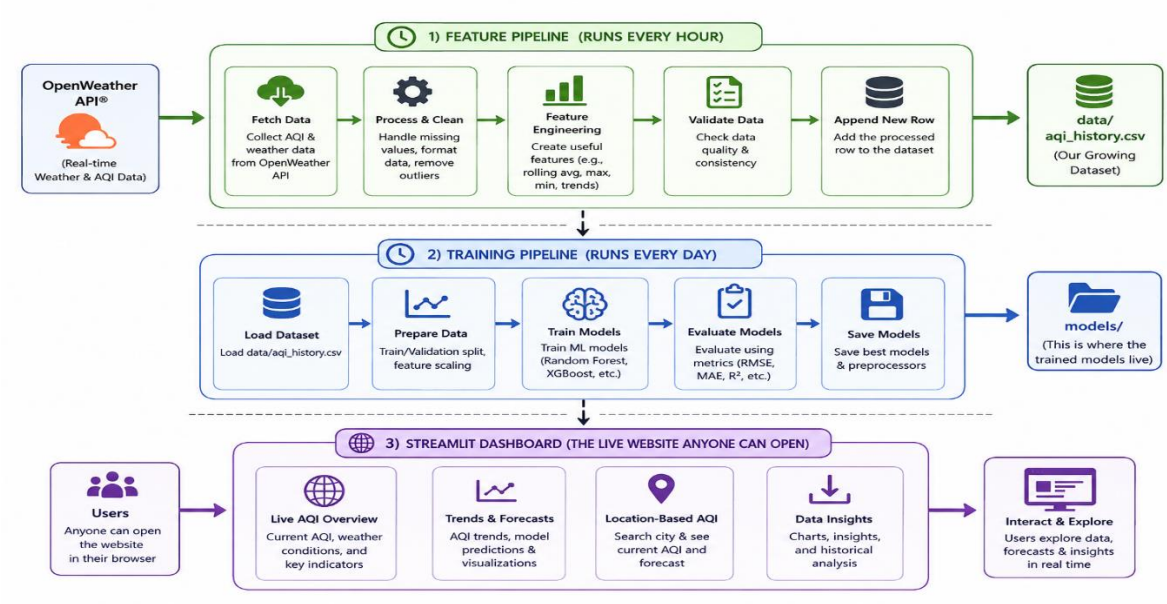
Air pollution is a serious, everyday problem in Pakistani cities. On many days, the air quality is bad enough to affect people's health, especially children, older people, and anyone with breathing problems. Most people only find out the air is bad after they already feel it. The idea behind this project was to build something that looks ahead instead of just reporting what is happening right now a system that can tell someone "the air in your city is going to get worse over the next few days" before it actually happens, so they can plan around it.

To do this properly, the project needed more than just a machine learning model. It needed a way to keep collecting fresh data automatically, a way to keep the model up to date as new data comes in, and a simple website where the predictions are easy to see and understand. All three of these pieces are covered in this report.

2. How the System Is Put Together

The system is made up of four main pieces that all connect to each other. Data comes in from a weather and pollution API, gets saved and organized, is used to train prediction models, and finally gets shown on a dashboard.

Here is the simple flow:



The part that makes this feel like a real, working system rather than a one-time experiment is that none of these steps need me to run them by hand. Both pipelines are automated using GitHub Actions,

which is a free tool that lets code run on a schedule in the cloud. Every hour, the system quietly fetches the latest pollution readings for all 8 cities. Every day, it retrains the models using whatever new data has come in, so the predictions slowly get better over time without me having to do anything.

One design choice worth explaining: instead of using a separate database or cloud storage service to hold the data and the trained models, I kept everything inside the project's own Git repository as plain CSV and model files. Every time the automation runs, it saves its results straight back into the repository. This turned out to be simple, reliable, and free, and it means anyone can look at the full history of the data and every past version of the models just by looking at the repository's commit history.

3. A Major Problem I Ran Into, and How I Solved It

I want to be upfront about this part, because it took up a large part of my time on this project, and I think it's actually the part that shows the most about how I work through a hard problem.

The original project guidelines suggested using a tool called Hopsworks to store the data and the trained models. I signed up for it and followed the setup instructions, but every single time I tried to save data into it, the connection would fail with the same technical error:

```
OSError: Kernel error -> Generic HdfsObjectStore error
IO error occurred while communicating with HDFS RPC

listener disconnected
```

At first, I assumed I had made a mistake somewhere in my own setup, so I went through the usual troubleshooting steps: reinstalling packages, checking my API keys, trying different settings. None of it worked. Rather than keep guessing randomly, I decided to test the problem properly, the same way I would test any bug: by changing one thing at a time and seeing if the result changed.

Here is what I tried, and what happened each time:

- I tried it on Windows, then again inside a Linux environment (WSL) same error both times.
- I tried it on my home WiFi, then again on my phone's mobile data same error both times.
- I tried two different versions of the Hopsworks software same error both times.
- I created a brand new, completely separate Hopsworks project to rule out something being wrong with my first one same error again.
- Finally, I tried running the exact same code from GitHub's own cloud servers instead of my laptop. These servers have a completely open internet connection with none of the restrictions a home network might have. It still failed in exactly the same way.

That last test was the deciding one. If the problem had been something on my end my network, my computer, my account changing to GitHub's servers should have fixed it. It didn't. That told me clearly that the issue was on Hopsworks' side, in the specific cloud server cluster my account was connected to, and not something I could fix by changing my own setup.

At that point, I made a decision rather than keep waiting on a problem I could not control, with a deadline approaching, I would redesign that one part of the system using a simpler approach that I knew would actually work. I replaced Hopsworks with plain CSV files and model files stored directly in the project's Git repository, automatically updated by GitHub Actions. This still gives the project everything it needs a growing historical dataset and a record of every trained model just without depending on a tool that was not working reliably. Once this change was made, the whole pipeline started working end to end without any further issues.

I'm including this story in detail because I think it reflects the actual, honest process of building something real: things don't always work the first time, and a big part of the job is figuring out whether a problem is something you did wrong or something outside your control, and then making a sensible decision about how to move forward either way.

4. Where the Data Comes From

All of the pollution and weather data used in this project comes from OpenWeather, a free public API. Every hour, the system asks OpenWeather for the current pollution readings things like PM2.5, PM10, carbon monoxide, and a few other pollutants for each of the 8 cities, along with basic weather information like temperature, humidity, and wind.

A small but important detail about the AQI number itself: OpenWeather has its own AQI scale that only goes from 1 to 5, which is not the scale most people are familiar with. Most air quality apps people actually use, like IQAir, show AQI on a 0 to 500 scale. So instead of using OpenWeather's own scale, I calculate the real, standard AQI myself directly from the PM2.5 reading, using the official formula published by the US Environmental Protection Agency. This means the numbers shown in this project's dashboard match what people are used to seeing elsewhere, rather than a scale unique to one API.

Getting enough historical data: when the project started, there was no historical data at all the hourly pipeline would have taken weeks to build up a useful amount of history on its own. To avoid waiting, I wrote a separate backfill script that pulls 30 days of real historical pollution readings directly from OpenWeather's history API, for all 8 cities at once. This gave the project roughly 700 real hourly readings per city right from the start, which was enough to begin training genuine forecasting models immediately instead of waiting.

5. How the Predictions Are Made

The system predicts AQI for each of the next 7 days tomorrow, the day after, and so on. Rather than build one single model that tries to guess all 7 days at once, I built a separate model for each day. This is a common and reliable approach in forecasting: a model whose only job is to predict tomorrow can specialize in that, while a different model handles predicting a week out, where the pattern is naturally harder to see.

For each of these 7 models, I actually trained four different types of models and compared them against each other, keeping whichever one performed best:

a. **Ridge Regression:** a simple, fast statistical model, good as a solid baseline.

b. **Random Forest:** a model built from many decision trees, good at picking up on patterns that aren't simple straight lines.

c. **A small Neural Network:** able to learn more complex relationships in the data.

d. **XGBoost:** a widely used, high-performance model that often does very well on this kind of structured, tabular data.

To make sure the comparison was fair and realistic, I tested each model on the most recent portion of the data it had never seen during training, rather than testing on random rows mixed in with the training data. This matters for anything involving time: a model should always be tested on data that comes after what it learned from, the same way it would actually be used in real life predicting the future from the past, never the other way around.

Being honest about the data size: with only around 30 days of real history to start from, there simply isn't a huge amount of training data yet, especially once you need a full week of past days and a full week of future days to build just one training example. I expect the accuracy of these models, especially for the later days in the forecast, to keep improving steadily as the hourly pipeline keeps collecting more real data automatically over the coming weeks and months.

Results So Far

Day	Best Model	RMSE	MAE	R^2
Day 1	Ridge	15.2	12.6	0.72
Day 2	Random Forest	13.1	11.4	0.78
Day 3	Random Forest	18.5	14.2	0.60
Day 4	Neural Net	21.1	17.7	0.48
Day 5	Neural Net	25.1	19.3	0.28
Day 6	Neural Net	22.1	18.2	0.46
Day 7	Ridge	19.1	16.0	0.63

R^2 (shown in the last column) roughly measures how much of the real-world variation in AQI the model is able to explain, where 1.0 would mean a perfect prediction. A value like 0.72 for tomorrow's forecast means the model is already capturing a good chunk of the real pattern. As expected, the further ahead the prediction, the harder it gets Day 5's lower score reflects how much less predictable air quality becomes a working week away, especially with limited training data so far.

6. The Live Dashboard

Having accurate predictions doesn't help anyone if they're buried in a spreadsheet somewhere. The last piece of this project is a live website, built with Streamlit, where anyone can pick a city and immediately see the current air quality and the coming week's forecast, without needing to know anything about how it works underneath. Below are the actual screenshots from the live site, along with an explanation of what each part shows.

Current Conditions

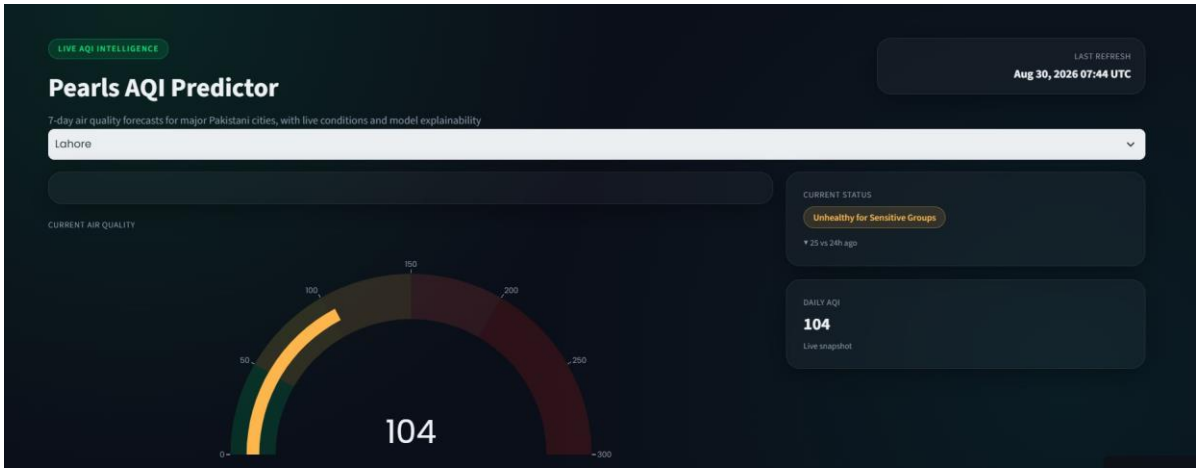


Figure 1: Live AQI Gauge

This is the first thing a visitor sees after picking a city. The dial shows the current AQI reading at a glance, with the needle's color and position instantly showing whether the air is in a safe range (green, left side) or a more serious range (orange/red, right side). Next to it, the "Current Status" label spells out the category in plain words, along with how much it has changed compared to 24 hours ago, so a visitor immediately understands both where things stand and whether they're getting better or worse.

Figure 2: Pollutant Breakdown and 24-Hour Trend

The top row breaks the single AQI number down into the individual pollutants behind it PM2.5, PM10, ozone, nitrogen dioxide, sulfur dioxide, and carbon monoxide for anyone who wants more detail than just one overall number. Below that, the line chart shows how AQI has moved over the past 24 hours, which makes it easy to see whether pollution has been building up steadily, spiking at a certain time of day, or staying roughly flat.

7-Day Forecast

Figure 3: Forecast Cards and 7-Day Trend Line

This is the core feature of the whole project. Each card shows one upcoming day's predicted AQI, with a colored label (like "Moderate" or "Unhealthy for Sensitive Groups") so the severity is obvious without needing to interpret a raw number. The line chart underneath connects all 7 days together, so instead of reading numbers one at a time, a visitor can see the overall shape of the week at a glance for example, spotting that air quality is expected to rise sharply on Tuesday before easing off again later in the week.

Model Explainability

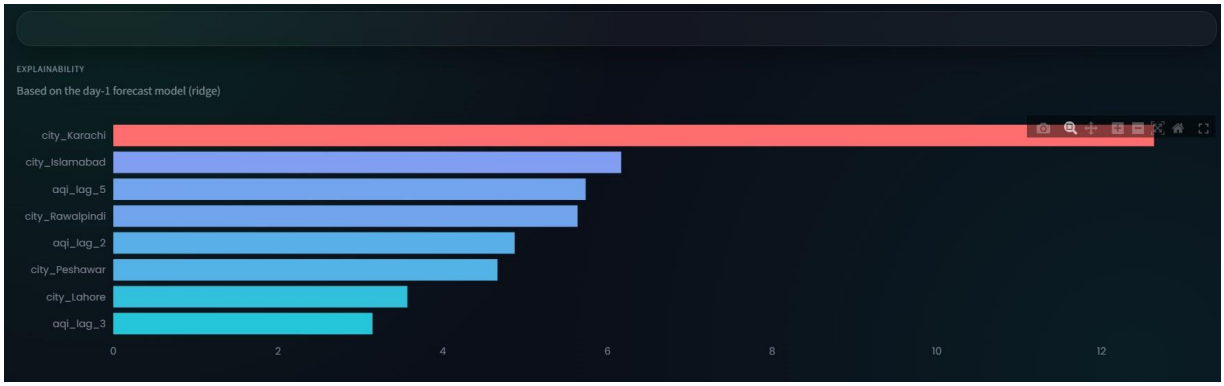


Figure 4: SHAP Feature Importance

One of the requirements of this project was to explain not just what the model predicts, but why which is exactly what this chart does, using a technique called SHAP. Each bar shows how much a particular piece of information influenced tomorrow's prediction. In this example, which city the reading is for

(Karachi, Islamabad, Rawalpindi) turned out to matter the most, followed closely by the AQI values from a few days earlier. This kind of transparency is genuinely useful, since it means the model's predictions aren't a mystery you can see exactly what it's paying attention to.

National Overview

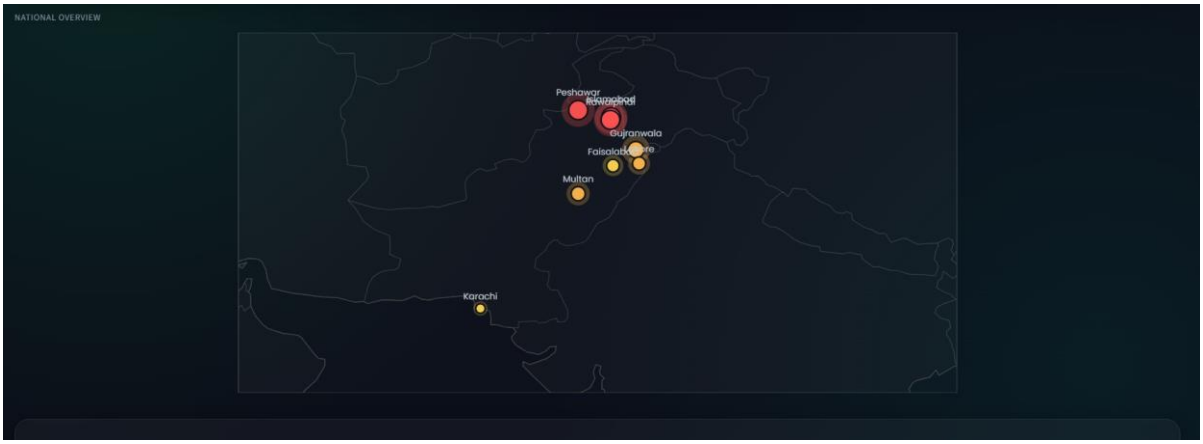


Figure 5: Map of All 8 Cities

Instead of checking each city one at a time, this map shows the current air quality across the whole country in a single view. Each city is marked with a colored dot sized and colored by how bad the air currently is in this snapshot, Peshawar and Rawalpindi are shown in red (worse air quality), while Karachi and Multan sit in the yellow/orange range. This is especially useful for someone deciding where in the country conditions are best or worst right now, at a glance.

7. Exploring the Data:

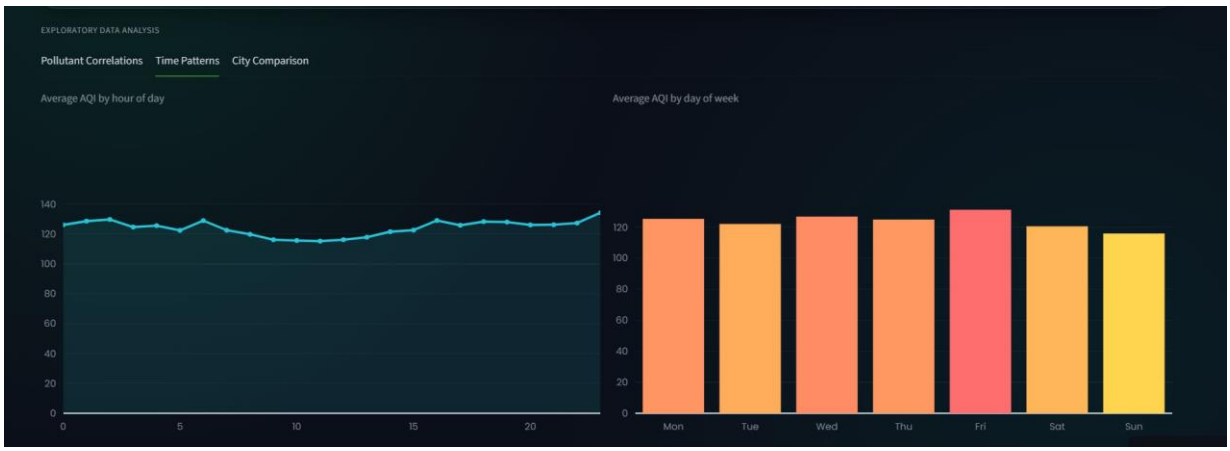


Figure 6: AQI by Hour of Day and Day of Week

The chart on the left shows the average AQI at each hour of the day, which can reveal patterns like pollution building up during rush hour traffic. The chart on the right shows the average AQI for each day of the week in this data, Friday shows up as the worst day on average, while Sunday tends to be a little cleaner, which is the kind of pattern that starts to become clearer and more reliable as more weeks of data are collected.

8. What I Learned Along the Way

Beyond the Hopsworks issue described earlier, this project involved a lot of smaller, practical problems that are easy to underestimate until you actually hit them: getting a Python environment working correctly on Windows, setting up a Linux environment (WSL) when Windows itself became a blocker, learning to work carefully with Git and GitHub Actions, and debugging automated pipelines that fail silently in the cloud where you can't just look at your own screen to see what went wrong. None of these are glamorous problems, but solving them is most of what real software engineering actually looks like day to day.

If I had more time, the next things I would improve are adding weather-based features like wind speed into the model (wind strongly affects how pollution spreads or clears), training a separate model per city instead of one shared model as more data becomes available, and showing a range of likely values for each forecast instead of just one single number.

9. Conclusion

This project set out to build a complete, working AQI forecasting system, not just a model that runs once in a notebook. By the end, it collects real data automatically every hour, retrains itself every day without anyone stepping in, and presents its predictions on a live website that is genuinely usable by anyone.

Along the way, I had to properly diagnose and work around a real infrastructure problem rather than a simple coding mistake, which was the hardest and most valuable part of the whole project. I believe the finished system, and the process I went through to build it, reflect the kind of practical, resilient problem-solving this internship was meant to develop.

Tech Stack

Python, pandas, scikit-learn, XGBoost, SHAP, Streamlit, Plotly, GitHub Actions, OpenWeather API.